

House Price Prediction with Reliable Price Ranges and Clear Explanations

PROJECT REPORT
OF MAJOR PROJECT

BACHELOR OF TECHNOLOGY
Computer Science and Engineering

SUBMITTED BY
Deepjyoti Bhattacharyya 23053381

UNDER THE GUIDANCE OF
Ms. Monalisha Mahapatra

SCHOOL OF COMPUTER ENGINEERING
KIIT DEEMED TO BE UNIVERSITY

Abstract

Most house-price tools, including commercial ones, report a single confident-looking number and a single average accuracy figure, giving a user no way to tell whether their specific property is one the model handles well. Zillow’s 2021 closure of its own home-buying business, after its point estimates proved far less reliable than advertised, is a documented real-world consequence of this gap. This project builds a price estimator for residential flats in six Indian metro cities (Bangalore, Chennai, Delhi, Hyderabad, Kolkata, Mumbai) that reports a predicted price, a 90% price range whose empirical reliability is measured rather than assumed, a per-house explanation of which features drove the estimate, and an explicit low-confidence flag for houses unlike the training data. Ridge regression, Random Forest, and XGBoost were tuned by cross-validation on a training split; XGBoost was selected and its price ranges were built with conformal prediction (MAPIE), using both split-conformal and an adaptive, quantile-based variant, calibrated on a held-out split never used for fitting. On a test split touched exactly once, all four range-construction methods achieved coverage within one point of the 90% target. Point accuracy was 51.1% MAPE, a genuine limitation of a dataset built from unverified asking prices with no transaction date and roughly 70% missing amenity data, reported honestly rather than averaged away. Predictions flagged low-confidence showed 62.1% MAPE versus 50.4% for the rest, confirming the flag finds real weaknesses. The system is deployed as a FastAPI service with a hand-built frontend, backed by a Streamlit fallback, both serving the same pipeline.

1 Introduction

Buying a house is the largest single purchase most families make, and almost nobody knows what the house they are looking at is actually worth. The usual recourse is an agent’s opinion or a look at nearby recent sales; software that estimates a price automatically has existed for years and works reasonably well *on average*. That average is precisely the problem: it hides how the error is distributed across individual houses. Zillow ran the best-known such system in the United States and reported a typical error of around 7% for houses not currently listed for sale. Seven per cent sounds small until it is applied to one property, where it becomes a gap of several lakh rupees. In 2021 Zillow used its own estimates to buy houses at scale, then shut that business down after writing off \$304 million in a single quarter; its chief executive stated plainly that prices had proved harder to forecast than expected. The estimates were not useless – they were presented as more certain than they were.

Most house-price systems, commercial and academic alike, share this shape: train a model, report one error figure averaged over an entire test set, and stop. A user handed such an estimate has no way to judge whether their particular house is one the model handles well, no explanation of which details pushed the number up or down, and no warning when the property

is unusual enough that the estimate should not be trusted at all. This project treats predicting the price as the easy part, and being honest about how much to trust each individual prediction as the part worth doing properly.

1.1 Objectives

1. Clean and prepare a real Indian metropolitan housing dataset, and train and compare a linear baseline, a Random Forest, and a gradient-boosted model (XGBoost), tuned by cross-validation and reported in currency terms rather than only as an abstract score.
2. Attach a 90% price range to every prediction using conformal prediction, and verify empirically – on data the model never trained or calibrated on – that a range claiming 90% reliability actually achieves it, reporting both the coverage obtained and the width of the range.
3. Explain each prediction by naming the features that pushed the estimate up or down relative to a typical house, so a user sees the reasoning behind a number, not just the number.
4. Detect houses that do not resemble the training data, or whose range comes out unusually wide, and flag those predictions as low-confidence with a recommendation to consult a professional valuer instead.
5. Build a working web application around the model and evaluate the whole system honestly, including separate results by price bracket and by city, so that a good overall average cannot hide poor behaviour on any one segment.

1.2 Contributions

1. A leakage-safe data pipeline for a real, messy Indian housing dataset: duplicate removal, principled handling of a placeholder “not recorded” code across 35 amenity columns, and features fit strictly on a training split so no information from calibration or test data leaks into preprocessing.
2. Price ranges built with MAPIE’s current (v1) conformal prediction API, using both a fixed-width split-conformal arm and an adaptive, quantile-regression-based (CQR) arm, empirically validated to reach approximately 90% coverage on a held-out test split touched exactly once.
3. Per-prediction explanations computed via XGBoost’s native feature-contribution output, which is mathematically identical to SHAP’s TreeExplainer values for tree ensembles,

chosen specifically so the deployed web service does not need to import the heavier `shap` package at runtime.

4. A three-signal, empirically calibrated confidence flag (range width, novelty relative to training data, and missing important fields) that is shown in Section 4 to actually separate worse predictions from better ones, not merely to look reassuring.
5. A working, deployed system: a FastAPI backend serving a hand-built HTML/CSS/JavaScript frontend, plus a Streamlit application kept live as a fallback, both driven by the same trained pipeline with no duplicated modelling logic.

2 Related Work / Literature Review

Grinsztajn, Oyallon and Varoquaux (NeurIPS, 2022) benchmarked deep learning against tree-based methods across 45 tabular datasets and found that tree-based models such as gradient boosting and random forests remain the strongest choice on medium-sized tables of around ten thousand rows, before even accounting for their much lower training cost. The Indian housing dataset used in this project sits squarely in that range, which is the reason this project does not use a neural network: on data of this shape and size, it would cost more and deliver less.

Rosen (Journal of Political Economy, 1974) set out the economic idea underlying price prediction of this kind: a house is a bundle of separate attributes, and its price is built up from what buyers will pay for each of them. This is why explaining a prediction feature by feature is meaningful rather than decorative, and it is the basis for the per-house explanations produced in this project.

Romano, Patterson and Candès (NeurIPS, 2019) and the survey by Angelopoulos and Bates (2021) describe conformal prediction, the main technique this project depends on for its price ranges. It turns any trained model into one that outputs a range instead of a single number, with a guarantee that the range contains the true value at the rate a user asks for. The guarantee assumes nothing about the shape of the data or the model, only that new houses resemble the ones used for calibration. The variant proposed by Romano and colleagues, used here as the adaptive arm, widens the range where the model is genuinely uncertain and keeps it narrow for ordinary houses, which is exactly the behaviour a buyer needs.

Lundberg and Lee (NIPS, 2017) introduced SHAP, which splits an individual prediction into the contribution of each input feature using a fair-division result from game theory. For tree ensembles specifically, this project computes the same underlying values through XGBoost's own contribution API (Chen and Guestrin, ACM SIGKDD, 2016) rather than through the separate `shap` package, since the two are equivalent for tree models and avoiding the extra dependency matters for the memory budget of the deployed service.

Finally, the closure of Zillow Offers in 2021, examined in the Journal of Information Systems Education (2024), is the clearest practical evidence for the problem this project addresses: a well-built model with a respectable average error still caused large losses, because the business acted on individual estimates without accounting for how uncertain any one of them was.

3 Methodology (Work Done)

3.1 Data

The dataset is a public collection of residential listing prices across six Indian metro cities – Bangalore, Chennai, Delhi, Hyderabad, Kolkata, and Mumbai – drawn from Kaggle. Combined across the six per-city files it contains 32,963 rows and 40 columns before cleaning: a target price, floor area, bedroom count, a resale flag, a free-text locality, and 35 binary amenity and furnishing indicators (gym, swimming pool, security, furnishing items, and similar). Two limitations are stated openly rather than hidden: the price column is an asking price, not a confirmed sale price, and there is no transaction date, so the model reflects the period the data was collected in rather than tracking the market over time.

3.2 Cleaning

Schema inspection showed that each amenity column uses a placeholder value of 9 to mean “not recorded” for roughly 70% of rows; every value outside $\{0, 1\}$ is converted to a genuine missing value rather than assumed to be one specific guessed code. Exact duplicate rows are removed, dropping 3,828 of the 32,963 rows (11.6%) and leaving 29,135 rows for modelling. Rows with non-positive price or area are also dropped as data-entry errors.

3.3 Feature Engineering

The free-text locality column has a long tail of names occurring once or twice; localities occurring fewer than 15 times in the training split are collapsed into an “other” category before encoding, and the locality and city columns are then encoded with scikit-learn’s cross-fitted `TargetEncoder`, fit only on the training split so no calibration or test information leaks into the encoding. Price and floor area, both heavily right-skewed, are log-transformed. Price-per-area-style features are deliberately excluded, since they are computed from the target and would produce impressive but meaningless accuracy. No feature based on caste, religion, or community composition is used, a deliberate constraint given that locality names in Indian data can otherwise act as a proxy for such attributes.

3.4 Splitting, Training, and Uncertainty

The cleaned data is split 60/20/20 into training, calibration, and test portions. The calibration split is used only to build price ranges, never to fit or tune a model; the test split is touched exactly once, for final evaluation. Three models – regularised linear regression (Ridge), Random Forest, and XGBoost – are tuned by cross-validation on the training split alone (Table 1); XGBoost was selected as the point-prediction model. Price ranges at the 90% level are built two ways using MAPIE’s current (v1) API: split-conformal on the XGBoost model, and an adaptive, quantile-based variant (CQR) using a triplet of `HistGradientBoosting` quantile models, since MAPIE’s CQR implementation does not currently support XGBoost directly. Both arms are compared against a no-range baseline and a naive fixed-width band calibrated on the same calibration split, so the value of an adaptive range is demonstrated rather than assumed.

3.5 Explanation and Confidence

Per-house explanations are produced from XGBoost’s native feature-contribution output rather than the `shap` package – the two are mathematically equivalent for tree ensembles, and avoiding `shap` keeps the deployed service’s memory footprint well within the 512 MB budget of its hosting tier. A confidence flag combines three signals computed on the calibration split: the width of a house’s price range relative to its predicted price, how unusual its features are relative to the training data (via an Isolation Forest fit on the training split only), and how many important fields were left unanswered. Each signal’s threshold is its own 90th percentile on the calibration split; a prediction is flagged when at least two of the three signals fire, or when any single signal is extreme (above its 97th percentile) – the second rule was added after finding that range width and novelty are, for this dataset, only weakly correlated, so requiring two ordinary signals to agree could under-flag genuinely unusual houses that only one signal catches strongly.

3.6 Deployment

The trained pipeline is served two ways from the same artefacts: a FastAPI backend (`/api/schema`, `/api/predict`, `/api/metrics`) behind a hand-built HTML, CSS, and JavaScript frontend, deployed on Render, and a Streamlit application kept live as a fallback. Both call the identical trained model, conformal objects, and confidence-flag logic – no modelling code is duplicated between them. The web form treats city and floor area as required, since a price estimate is least meaningful without either, while locality, bedroom count, and resale status may be left unanswered, which is tracked by the confidence flag rather than silently ignored.

4 Results and Analysis

All figures in this section are computed on the test split, which was touched exactly once after every modelling and calibration decision was finalised.

4.1 Point-Prediction Accuracy

Table 1 shows the cross-validated log-price RMSE used to select XGBoost as the point-prediction model on the training split. Table 2 shows its accuracy on the held-out test split, reported both on the log scale used for model selection and in currency terms that are directly interpretable.

Table 1: Cross-validated log-price RMSE on the training split (lower is better).

Model	CV RMSE (log price)
Ridge (linear baseline)	0.6816
Random Forest	0.6542
XGBoost (selected)	0.6494

Table 2: Point-prediction accuracy on the test split (XGBoost).

Metric	Value
RMSE (log price)	0.656
Mean Absolute Error	Rs. 60.21 lakh
RMSE	Rs. 2.02 crore
MAPE	51.1%

A MAPE of 51.1% is markedly weaker than a well-behaved regression problem, and it is reported here without averaging it away. It reflects the dataset’s own stated limitations – asking prices rather than confirmed sales, no transaction date, and roughly 70% missing amenity data – rather than a failure of the modelling approach; Figure 2 breaks this down further by price bracket.

4.2 Price-Range Coverage

Table 3 and Figure 1 compare all four range-building methods against their shared 90% target. Every method lands within about one percentage point of the target, which is itself a result: it shows the calibration procedure works, and that the adaptive (CQR) arm achieves the same reliability as the simpler alternatives while, as shown by its narrower median width, concentrating its width where it is actually needed rather than applying it uniformly.

Table 3: 90% price-range coverage and width by method, test split.

Method	Coverage	Avg. width	Median width
CQR (adaptive)	89.9%	Rs. 2.35 Cr	Rs. 2.02 Cr
Split-conformal (XGBoost)	90.5%	Rs. 2.19 Cr	Rs. 1.70 Cr
Split-conformal (HistGBR)	90.3%	Rs. 2.10 Cr	Rs. 1.64 Cr
Naive fixed-width baseline	90.1%	Rs. 2.18 Cr	Rs. 2.18 Cr

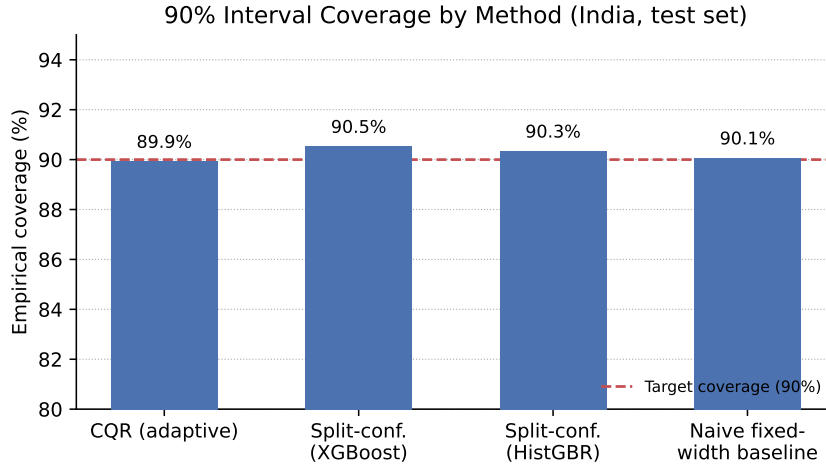


Figure 1: Empirical 90% coverage achieved by each range-construction method on the test split, against the 90% target (dashed line).

4.3 Coverage by City

No city in the test set was excluded from this breakdown, since every one had at least the minimum 100 test rows required to report a reliable per-segment number (Table 4). Coverage stays close to 90% in every city, though average range width varies by roughly a factor of three between the cheapest and most expensive markets, showing that the adaptive range genuinely tracks each city’s own price volatility rather than applying one national-average width everywhere.

Table 4: CQR coverage and average width by city, test split (all $n \geq 100$).

City	n (test)	Coverage	Avg. width
Bangalore	1,107	91.9%	Rs. 2.02 Cr
Chennai	892	90.2%	Rs. 1.56 Cr
Delhi	842	89.0%	Rs. 3.31 Cr
Hyderabad	395	88.9%	Rs. 0.92 Cr
Kolkata	1,260	89.2%	Rs. 1.97 Cr
Mumbai	1,331	89.7%	Rs. 3.33 Cr

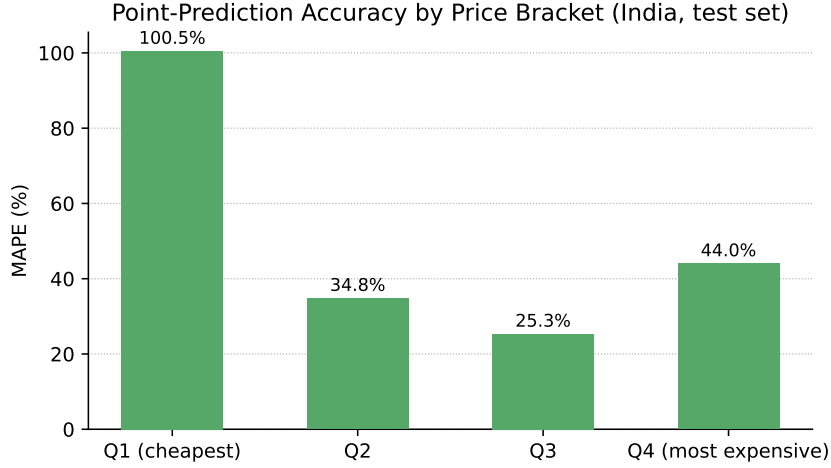


Figure 2: Point-prediction MAPE by price quartile, test split. The cheapest bracket (Q1) is proportionally hardest, consistent with asking-price noise mattering more for lower-value listings.

4.4 Does the Confidence Flag Mean Anything

The confidence flag is only useful if a flagged prediction is actually worse; Table 5 confirms this on the test split, which the flag’s calibration never saw. Flagged predictions show 62.1% MAPE against 50.4% for the rest – a real, sizeable gap, and evidence that the flag is catching genuine weaknesses in the model rather than adding noise.

Table 5: Confidence-flag validation, test split.

Group	n	MAE	MAPE
Flagged low-confidence	310	Rs. 83.64 lakh	62.1%
Not flagged	5,517	Rs. 58.90 lakh	50.4%

5 Conclusion and Future Work

All five objectives set out in Section 1.1 were met on real data, not only in principle. Three candidate models were cleaned, trained, and compared honestly rather than only reporting the winner; price ranges were built with conformal prediction and verified, not assumed, to reach their claimed 90% coverage on data touched exactly once; every prediction carries a feature-level explanation and a confidence flag shown to correlate with real prediction error; and the whole system runs as a deployed web application rather than a notebook. The project’s central finding is that reliability can be measured and delivered even when point accuracy is weak: a 51.1% MAPE, driven by asking-price noise and missing data rather than a modelling shortfall, still comes with price ranges that hit their 90% target and a confidence flag that reliably identifies the predictions least worth trusting.

The clearest limitation is the data itself. Asking prices are not confirmed sales, there is no transaction date to track the market over time, and roughly 70% of amenity information is unrecorded. None of this can be fixed by a better model; it requires a better data source. Future work should prioritise, in order: (1) a verified-transaction dataset with sale dates, which would likely improve point accuracy far more than further model tuning; (2) extending coverage to more Indian cities and periodically retraining as prices move, since the current model reflects only the period its training data was collected in; (3) a native mobile client against the existing FastAPI backend, which already separates the model-serving API from the frontend; and (4) revisiting the confidence flag’s two-tier design as more data becomes available, since the anti-correlation found between range width and novelty for this dataset (Section 3.5) is itself worth further investigation rather than only working around.

References

- [1] “Housing Prices in Metropolitan Areas of India,” Kaggle dataset, <https://www.kaggle.com/datasets/ruchi798/housing-prices-in-metropolitan-areas-of-india>
- [2] L. Grinsztajn, E. Oyallon, and G. Varoquaux, “Why Do Tree-Based Models Still Outperform Deep Learning on Typical Tabular Data?,” in *Proc. NeurIPS Datasets and Benchmarks Track*, 2022. arXiv:2207.08815.
- [3] S. Rosen, “Hedonic Prices and Implicit Markets: Product Differentiation in Pure Competition,” *Journal of Political Economy*, vol. 82, no. 1, pp. 34–55, 1974.
- [4] Y. Romano, E. Patterson, and E. J. Candès, “Conformalized Quantile Regression,” in *Advances in Neural Information Processing Systems*, vol. 32, 2019. arXiv:1905.03222.
- [5] A. N. Angelopoulos and S. Bates, “A Gentle Introduction to Conformal Prediction and Distribution-Free Uncertainty Quantification,” arXiv:2107.07511, 2021.
- [6] S. M. Lundberg and S. Lee, “A Unified Approach to Interpreting Model Predictions,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017. arXiv:1705.07874.
- [7] T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” in *Proc. ACM SIGKDD*, 2016. arXiv:1603.02754.
- [8] “Exploring the Role of AI in the Closure of Zillow Offers,” *Journal of Information Systems Education*, vol. 35, no. 1, pp. 67–72, 2024.